

Modelado de datos

Entidad ← Objeto o caso del mundo real que tiene existencia independiente

Atributo ← Propiedades o características de una entidad ← La clave es uno o varios atributos

Relación ← Asociación entre dos o más entidades
↑ Pueden ser reflexivas

Dominio ← Conjunto de valores permitidos para un atributo

Cardinalidad ← Número de ocurrencias de una entidad que pueden estar asociadas a otra entidad en una relación.

Clave ← Atributo que identifica de manera única a cada ocurrencia dentro de una entidad. ← Si una entidad no tiene clave única se considera una entidad débil.

Entidad débil ← Entidad que depende de otra

→ Por identificación: Requieren la clave de una entidad fuerte. Su existencia está sujeta a la relación con la otra entidad. ← Clave en subgrupo discontinuo.

→ Por existencia: Dependen de la existencia de la otra entidad.

Condiciones del modelado

- Sin filas duplicadas
- No se permiten valores multivaluados

Normalización

↑ Pretende eliminar las anomalías en la inserción borrado o actualización de datos

1FN ← Todas las columnas de la tabla con valores atómicos.

2FN { - 1FN
- Los atributos que no son clave deben depender por completo

3FN { - 2FN
- No debe haber dependencias funcionales indirectas entre columnas no clave y la clave primaria.

Paso a tablas

N:M

Se transforma en una tabla que incluye los atributos de la relación y las claves de las entidades relacionadas.

1:N

La entidad con cardinalidad 1 migra sus claves a la entidad con cardinalidad N.

1:1

Misma cardinalidad $\begin{cases} - 1 \text{ se fusionan junto los atributos de la relación.} \\ - 0 \text{ se añade la clave de una tabla a la otra indistintamente} \end{cases}$

Diferente cardinalidad $\leftarrow$ la clave de la entidad 1 va a la de 0.

Álgebra Relacional

Lenguaje procedimental

Selección σ $\leftarrow$ Filtra filas según una condición

$\sigma_{\text{ciudad} = \text{Madrid}} \text{ Clientes}$

Proyección π $\leftarrow$ Selecciona columnas específicas de una tabla

$\pi_{\text{nombre, edad}} \text{ Clientes}$

Producto cartesiano $\times$ $\leftarrow$ Combina todas las filas de dos tablas

$\text{Clientes} \times \text{Pedidos} \leftarrow$ crea una tabla con todas las combinaciones

Unión $\cup$ $\leftarrow$ Combina filas de dos tablas con la misma estructura

$\pi_{\text{nombre}} \text{ Cliente} \cup \pi_{\text{nombre}} \text{ Empleado}$

Diferencia $-$ $\leftarrow$ Retorna las de la primera tabla que no están en la segunda.

$\pi_{\text{id}} \text{ Clientes} - \pi_{\text{id}} \text{ Pedidos}$

Intersección $\cap$ $\leftarrow$ Retorna filas que están en ambas tablas $\leftarrow$ con la misma estructura.

$\pi_{\text{id, nombre}} \text{ Clientes} \cap \pi_{\text{id, nombre}} \text{ Compras}$

División : $\leftarrow$ Encuentra registros de una tabla que están relacionados con todos los registros de la otra.

$\text{grado}(R) > \text{grado}(S) \rightarrow \left(\pi_{\text{id_cliente, id_producto}} \text{Compras} \right) \div \left(\pi_{\text{id_producto}} \text{Productos} \right)$
Clientes que compraron todos los productos

Natural join $\bowtie$ $\leftarrow$ Combina tablas basándose en columnas con el mismo nombre.

$R \bowtie S = \sigma_{\neq}(R \times S)$

Clientes $\bowtie$ Pedidos

Outer join

$\bowtie_{\text{Left}}$ $\leftarrow$ Mantiene todas las filas de la tabla izquierda rellenando con NULL si no hay match

$\bowtie_{\text{Right}}$ $\leftarrow$ Mantiene todas las filas de la tabla derecha.

$\bowtie_{\text{Full}}$ $\leftarrow$ Mantiene filas de ambas tablas.

SQL

Triggers

before } - Insert
after } - Update
 - Delete

DELIMITER \$\$

CREATE TRIGGER

FOR EACH ROW

BEGIN

-- Acciones

END \$\$

DELIMITER;

trigger-name [BEFORE] [AFTER] [INSERT] [UPDATE] [DELETE] ON table

NEW.Columna $\leftarrow$ Se refiere a los nuevos valores durante INSERT o UPDATE

OLD.Columna $\leftarrow$ Se refiere a los valores anteriores durante UPDATE o DELETE.

Procedimiento → CALL nombre-procedimiento (parámetros)

```
DELIMITER ##  
CREATE PROCEDURE procedure_name ([IN]  
                                [OUT]  
                                [INOUT])  
BEGIN  
    -- Cuerpo del procedimiento [SIGNAL SQLSTATE]  
END ##  
DELIMITER;
```

Funciones → SELECT nombre-funcion

```
DELIMITER ##  
CREATE FUNCTION function_name (parámetros) RETURNS type_return  
DETERMINISTIC  
BEGIN  
    [DECLARE] ← Variables locales  
    [SET]  
    [RETURN]  
END ##  
DELIMITER;
```

Cursor → uso en procedimientos, funciones y triggers

```
DECLARE fin INT DEFAULT FALSE  
DECLARE cursor_name CURSOR FOR SELECT ... FROM ... WHERE ...  
DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = TRUE  
  
OPEN cursor_name  
  
REPEAT  
    FETCH cursor_name INTO [variables]  
    IF NOT fin THEN  
        -- Lógica del cursor  
    END IF  
UNTIL fin END REPEAT  
  
CLOSE cursor_name
```

Vistas $\rightarrow$ Select * from view-name

CREATE VIEW view-name AS

SELECT ... } Consulta

Anotaciones Hibernate

@Entity $\leftarrow$ Indica que la clase es una entidad persistente

@Table(name = " ") $\leftarrow$ Especifica el nombre de la tabla

@Id $\leftarrow$ Marca un campo como clave primaria.

@Column $\leftarrow$ Mapea un atributo como columna

@GeneratedValue $\leftarrow$ Autogeneración de claves

Relacion 1:N (Serie: Capítulos)

@OneToMany(mappedBy = "serie", cascade = CascadeType.ALL)
private Set<Capitulo> capitulos;

@ManyToOne

@JoinColumn(name = "serie_id")
private Serie serie;

Relacion N:M (Usuarios: Capítulos Vistos)

@ManyToMany

@JoinTable(name = "usuarios_capitulos_vistos")
private Set<Capitulo> capitulosVistos;

@ManyToMany(mappedBy = "capitulosVistos")
private Set<Usuario> usuarios;

